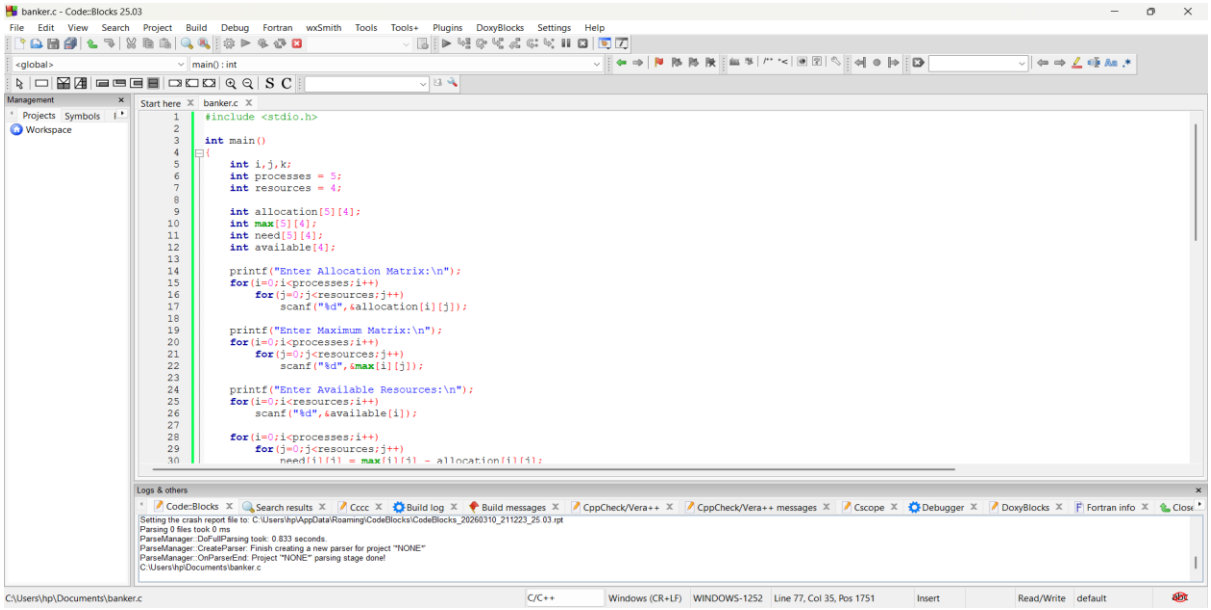
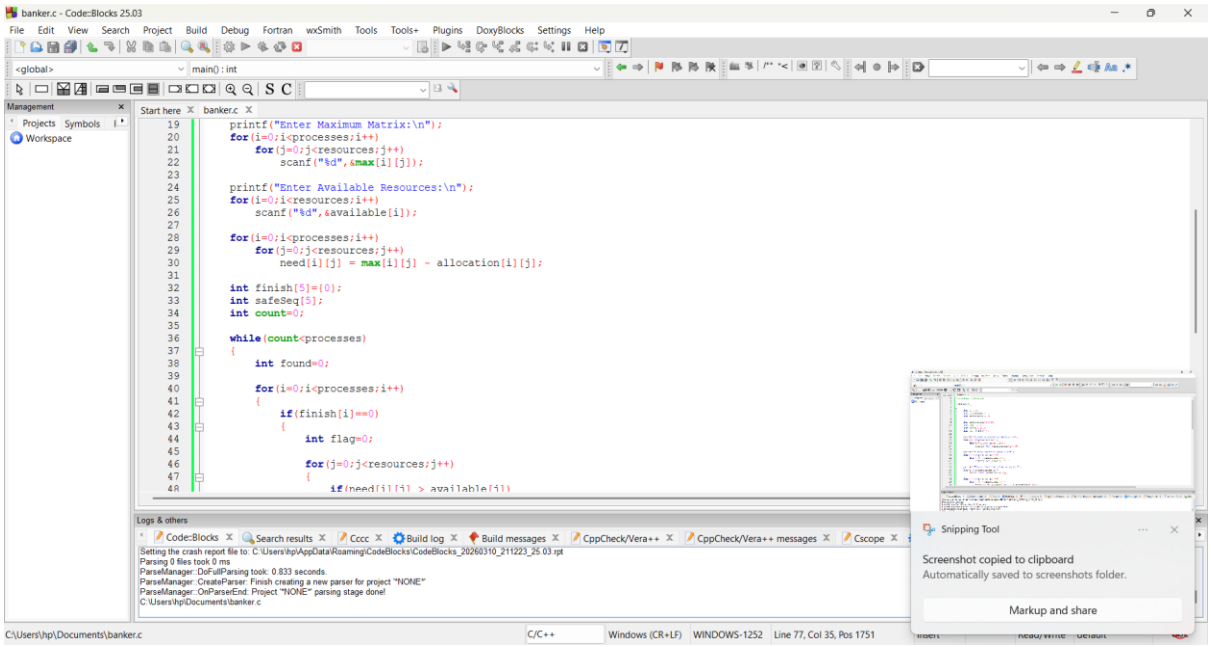
Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

INPUT:



```c
#include <stdio.h>

int main()
{
    int i,j,k;
    int processes = 5;
    int resources = 4;

    int allocation[5][4];
    int max[5][4];
    int need[5][4];
    int available[4];

    printf("Enter Allocation Matrix:\n");
    for(i=0;i<processes;i++)
        for(j=0;j<resources;j++)
            scanf("%d",&allocation[i][j]);

    printf("Enter Maximum Matrix:\n");
    for(i=0;i<processes;i++)
        for(j=0;j<resources;j++)
            scanf("%d",&max[i][j]);

    printf("Enter Available Resources:\n");
    for(i=0;i<resources;i++)
        scanf("%d",&available[i]);

    for(i=0;i<processes;i++)
        for(j=0;j<resources;j++)
            need[i][j] = max[i][j] - allocation[i][j];
```



```c
    printf("Enter Maximum Matrix:\n");
    for(i=0;i<processes;i++)
        for(j=0;j<resources;j++)
            scanf("%d",&max[i][j]);

    printf("Enter Available Resources:\n");
    for(i=0;i<resources;i++)
        scanf("%d",&available[i]);

    for(i=0;i<processes;i++)
        for(j=0;j<resources;j++)
            need[i][j] = max[i][j] - allocation[i][j];

    int finish[5]={0};
    int safeSeq[5];
    int count=0;

    while(count<processes)
    {
        int found=0;

        for(i=0;i<processes;i++)
        {
            if(finish[i]==0)
            {
                int flag=0;

                for(j=0;j<resources;j++)
                {
                    if(need[i][j] > available[j])
```

banker.c - Code::Blocks 25.03

```c
                for(j=0;j<resources;j++)
                {
                    if(need[i][j] > available[j])
                    {
                        flag=1;
                        break;
                    }
                }

                if(flag==0)
                {
                    for(k=0;k<resources;k++)
                        available[k]+=allocation[i][k];

                    safeSeq[count++]=i;
                    finish[i]=1;
                    found=1;
                }
            }
        }

        if(found==0)
        {
            printf("System is NOT in safe state");
            return 0;
        }
    }

    printf("Safe sequence is:\n");
```

banker.c - Code::Blocks 25.03

```c
                }

                if(flag==0)
                {
                    for(k=0;k<resources;k++)
                        available[k]+=allocation[i][k];

                    safeSeq[count++]=i;
                    finish[i]=1;
                    found=1;
                }
            }
        }

        if(found==0)
        {
            printf("System is NOT in safe state");
            return 0;
        }
    }

    printf("Safe sequence is:\n");

    for(i=0;i<processes;i++)
        printf("P%d ",safeSeq[i]);

    return 0;
}
```

OUTPUT:

```
C:\Users\hp\Documents\bank
Enter Allocation Matrix:
1 0 1 2
0 1 0 1
1 1 0 0
0 0 1 1
1 0 0 1
Enter Maximum Matrix:
2 1 1 2
1 2 1 2
1 1 1 1
1 1 2 2
2 1 1 1
Enter Available Resources:
1 1 1 1
Safe sequence is:
P0 P1 P2 P3 P4
Process returned 0 (0x0)   execution time : 75.961 s
Press any key to continue.
```